

The layer is a matrix multiply

On the first day of this course you ran a line called `X @ W + b` and I told you it was a version check. It was not. Today you find out what you actually computed in week one — and it is one complete layer of a neural network.

Unit 1 · Foundations

80 minutes

Lab 5 · a layer by hand

Paper before keyboard

[The dot product](#)[Matrix multiply](#)[Week one, explained](#)[A batch](#)[Nine weights](#)[What is missing](#)[Lab 5](#)[Before Tuesday](#)

PART 1 · 10 MINUTES

The dot product: multiply, then add up

One operation. Everything today is built from it, and so is every model in this course.

1

Take two lists of the same length. Multiply them element by element, then add the results into a single number.

the whole definition

PYTHON

```
u = np.array([1., 2., 3.])
v = np.array([4., 5., 6.])
```

```
u @ v      → 32.0
```

```
# by hand: 1×4 + 2×5 + 3×6 = 4 + 10 + 18 = 32
```

Two arrays go in, one number comes out. The `@` operator is Python's matrix-multiplication symbol; for two 1-D arrays it does exactly the multiply-and-add above.



Read it as "evidence, weighted"

Think of **u** as measurements and **v** as how much each measurement counts. The dot product is the total score. In session 2 you saw a spam filter counting word frequencies into an evidence table — that table is **v**, the email is **u**, and this single operation is the whole classifier. You have been looking at dot products since week one without a name for them.

PART 2 · 14 MINUTES

Matrix multiplication is many dot products at once

No new idea. Just bookkeeping about which row meets which column.

2

When **A @ B** has 2-D arrays on both sides, the entry in row *i*, column *j* of the result is the dot product of **row *i* of A** with **column *j* of B**. That is the entire rule.

ROW OF A MEETS COLUMN OF B, ONE DOT PRODUCT PER OUTPUT CELL

$$\begin{array}{|c|c|} \hline A(1,2) \\ \hline 1.0 & 2.0 \\ \hline \end{array} @ \begin{array}{|c|c|c|} \hline W(2,3) \\ \hline 0.5 & -1.0 & 2.0 \\ \hline 1.5 & 0.0 & -0.5 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline (1,3) \\ \hline 3.5 & -1.0 & 1.0 \\ \hline \end{array}$$

The highlighted cell:

row of A · column 0 of W = $1.0 \times 0.5 + 2.0 \times 1.5 = 0.5 + 3.0 = 3.5$

Three output cells, three dot products. Nothing else is happening.

The shape rule — the inner numbers must match

MEMORISE THIS LINE

$$(n, k) @ (k, m) \rightarrow (n, m)$$

The two inner *ks* must be equal — that is the length of the dot products. They then vanish, and the outer numbers survive. If you can look at two shapes and say the answer, you can debug three quarters of what goes wrong in unit 2.

practise reading the shapes

PYTHON

```
(4, 2) @ (2, 3)  → (4, 3)    # the 2s match and disappear
(3, 4) @ (4, 1)  → (3, 1)    # the 4s match
(2, 3) @ (2, 3)  → ValueError # 3 ≠ 2. no.
```



@ and * are different operators. Never confuse them.

`A * B` multiplies element by element and uses *broadcasting* — Friday's rules. `A @ B` is matrix multiplication and uses the inner-dimension rule above. They agree on almost nothing, and both will happily run on the same pair of arrays and give you different answers.

PART 3 · 16 MINUTES · THE POINT OF THE SESSION

What you actually ran in week one

Open your session 1 notebook. It is still there.

3

check_setup.py, from session 1 — unchanged

WEEK 1

```
X = np.array([[1.0, 2.0]])          # one example, two features
W = np.array([[0.5, -1.0, 2.0],
               [1.5, 0.0, -0.5]])    # 2 inputs → 3 hidden units
b = np.array([0.1, 0.1, 0.1])

out = X @ W + b
→ array([[ 3.6, -0.9,  1.1]])
```

You now have every piece needed to read that. Take it apart:

Symbol	Shape	What it is
X	$(1, 2)$	One example with two features. Rows are examples, columns are features.
W	$(2, 3)$	The weights . Column j holds the weights of hidden unit j — how much each input counts for that unit. Six numbers, because 2 inputs $\times$ 3 units.
b	$(3,)$	The biases . One per unit — a shift that moves the unit's threshold independently of the inputs. Broadcast across every row, by Friday's rules.
$X @ W + b$	$(1, 3)$	The output of three units, for one example.

EVERY NUMBER, BY HAND

three dot products, one per unit

ARITHMETIC

```
unit 0:  1.0×0.5  + 2.0×1.5  + 0.1  =  0.5 + 3.0 + 0.1  =  3.6
unit 1:  1.0×-1.0 + 2.0×0.0  + 0.1  = -1.0 + 0.0 + 0.1  = -0.9
unit 2:  1.0×2.0  + 2.0×-0.5 + 0.1  =  2.0 - 1.0 + 0.1  =  1.1

                → [ 3.6, -0.9, 1.1 ]
```

SO HERE IS WHAT THAT LINE WAS

$X @ W + b$ is **one complete layer of a neural network**.

Not an analogy. Not a simplified version. That is the operation, exactly as it runs inside every model in this course and inside the frontier models you used last week. The only thing separating that line from a working network is a detail we will add in session 11 — and the numbers, which right now I chose and which from session 10 the machine will choose for itself.



Back to the picture from the orientation deck

Each column of W is one artificial neurone: it collects every input, multiplies each by how much that input counts, adds them up, and adds its own bias. Many inputs, one output, a threshold in between. The biological picture you saw in the orientation talk, and this matrix, are the same object described two ways.

Four examples cost the same as one

Why the convention is rows-are-examples, and why nothing in your code has to change.

4

stack four examples as four rows — W and b untouched

PYTHON

```
Xb = np.array([[ 1.,  2.],
               [ 0.,  1.],
               [ 2.,  0.],
               [-1.,  3.]])      # shape (4, 2)
```

```
Xb @ W + b
→ array([[ 3.6, -0.9,  1.1],
         [ 1.6,  0.1, -0.4],
         [ 1.1, -1.9,  4.1],
         [ 4.1,  1.1, -3.4]])    # shape (4, 3)
```

The first row is identical to before — same example, same answer. The code did not change at all: $(4, 2) @ (2, 3) \rightarrow (4, 3)$, and $\mathbf{b}$ of shape $(3,)$ broadcasts across all four rows exactly as Friday's first table said it would.

⚡ This is the whole reason for the convention

Sixty thousand examples is $(60000, 784) @ (784, 128)$ — one call, no loop over examples, and the processor works on the whole block at once. Rows-are-examples is not an arbitrary style choice; it is what lets a batch cost barely more than a single case. When session 16 asks for a mini-batch of 32, it will be $X[i:i+32]$ — Tuesday's slicing, and nothing more.

Nine weights and four biases — where those numbers come from

The network from the orientation deck, counted from its shapes.

5

A network with 2 inputs, 3 hidden units and 1 output. Two layers, each one $@$ and a $+$:

the shape chain – the second layer is the same operation again

PYTHON

```
W1, b1 = shape (2, 3), shape (3,)      # inputs → hidden
W2, b2 = shape (3, 1), shape (1,)      # hidden → output

h = Xb @ W1 + b1      # (4, 2) @ (2, 3) → (4, 3)
o = h @ W2 + b2      # (4, 3) @ (3, 1) → (4, 1)

# four examples in, four predictions out
```

	Shape	Count	Meaning
W1	(2, 3)	6	One weight per connection from an input to a hidden unit
W2	(3, 1)	3	One weight per connection from a hidden unit to the output
b1	(3,)	3	One bias per hidden unit
b2	(1,)	1	One bias for the output unit
Total		9 weights · 4 biases	One weight per line, one bias per unit



Thirteen numbers here. Hundreds of billions there.

A frontier language model is this same pair of quantities — weights and biases — at a scale about ten billion times larger, arranged in more layers. Not a different kind of thing. When you hear "a 70-billion-parameter model", it means 70 billion entries in matrices exactly like W1, every one of them found by the procedure you will write in session 14.

What is still missing — and why it matters more than it looks

6

Two stacked layers, right now, are worth exactly one. Here is the proof.

run this — the result is not a rounding artefact

PYTHON

```
rng = np.random.default_rng(1)
A = rng.normal(size=(2, 3))
B = rng.normal(size=(3, 1))
x = np.array([[1., 2.]])

x @ A @ B          → [[3.1895]]    # two layers

C = A @ B           # collapse them into ONE matrix, shape (2, 1)
x @ C               → [[3.1895]]    # one layer — identical

np.allclose(x @ A @ B, x @ C) → True
```

A HUNDRED STACKED LAYERS WOULD GAIN YOU NOTHING

Two matrix multiplies in a row can always be replaced by a single matrix. So can a hundred. As written, "deep" learning would be exactly as powerful as one layer — a chain of straight lines is still a straight line.

The fix is one small non-linear step applied after each layer, called an **activation**. It is a single line of code, you meet it in session 11, and it is the entire reason depth means anything at all.

So the honest state of play: you can now compute a layer, you can batch it, and you know why stacking is currently pointless. Session 11 supplies the missing bend. Sessions 10 and 11 supply the thing that actually matters more — how the numbers in **W** stop being ones I chose and start being ones the machine found.

A layer by hand, then in one line

Lab 5 is a separate handout — `CSC4315_Lab05_A_Layer_By_Hand.html`. Paper first, then code, then check the two agree digit for digit. That order is the method for the entire rest of this course, and this is the session where it starts.



Same rules

Finish in the room, hand up after three minutes. Nothing in Lab 5 is a prerequisite for Tuesday — but this is the first lab where the hand computation genuinely matters, because session 13 asks you to do the same thing on a network four times the size.

Before Tuesday

1. **Finish Lab 5**, including the photograph of your paper working. The paper is the part I care about.
2. **Drill the shape rule until it is instant.** Say the result, or "error", for each: $(8,3)@(3,5) \cdot (5,5)@(5,1) \cdot (2,4)@(2,4) \cdot (1,7)@(7,7) \cdot (6,2)@(3,2)$. You should not need to think about these by Tuesday.
3. **Read one sentence and sit with it.** Every number in `W` was chosen by me, arbitrarily. Tuesday is about what it would mean for a machine to choose them instead — and it begins with a curve you already plotted in Lab 4.



Session 9 — Tuesday 06 October, 16:00

Slope, the chain rule, and gradient descent by hand. We minimise $f(x) = x^2$ with a pencil, five steps, then in three lines of code — and then watch what happens when the learning rate is too small, too large, and right. It is the last session of unit 1, and it is the one that makes training stop being magic.